Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Department of Information Technology

Course Code and Name: INSY 8311 – Database Development with PL/SQL

Assignment 4 (Project): The AUCA Registrar Module – Procedures and Functions

Released Date: Sunday,2nd November 2026

Deadline: Sunday, 15th November 2026, 11:59 PM

1. Purpose

The first three assignments were sets of separate questions. This one is a small project. You will build a working piece of software inside the database: a set of stored functions and procedures that the Registrar's Office can call instead of editing tables by hand. The main topic is Chapter 4 (procedures, functions, parameter modes IN / OUT / IN OUT, default values, positional and named notation, calling functions from SQL). The project also depends on what you learned before: SQL (Phase 0 and Phase 4), variables, IF, CASE and loops (every phase) and cursors (Phase 3).

You will use the same schema you already set up in your XEPDB1 database as the **lecturer** user: **STUDENTS**, **COURSES**, **ENROLLMENTS**, **EMPLOYEES**, and **DEPARTMENTS**, plus one new table that you create in Phase 0.

2. The Scenario

The Registrar's Office at AUCA currently manages enrollments and grades by running UPDATE and INSERT statements by hand. Last semester this caused several problems: a student was enrolled twice in the same course, a course was filled beyond its capacity, a score of 105 was saved, and nobody could say who had changed a student's GPA or when.

You have been hired as the database developer to build the **Registrar Module**. From now on, staff will never touch the tables directly: they will call your functions and procedures, which apply the university's rules every time and record every change in a log. At the end of the project you will run a "Registration Day" test script that proves the module works, and you will present your results in your report.

Business rules the module must enforce

Rule	Description
R1	Only students whose status is 'Active' may enroll in a course.
R2	A student cannot be enrolled twice in the same course.

Rule	Description
R3	The number of Active enrollments in a course may not exceed its max_students.
R4	500-level courses (the first digit of the number in the course code is 5, e.g. CS501) require a GPA of at least 3.0.
R5	A score must be between 0 and 100 and is rounded to the nearest 0.5 before it is stored (e.g. 87.3 becomes 87.5).
R6	Letter grades: 90 and above A, 80 to below 90 B, 70 to below 80 C, 60 to below 70 D, below 60 F.
R7	GPA = average of the student's graded scores / 25, rounded to 2 decimal places and never above 4.00. It is only recalculated when the student has at least one graded course.
R8	Every enrollment, rejected enrollment, score change and GPA change is written to REGISTRAR_LOG, with the database user (USER) and the time (SYSDATE).
R9	Functions and procedures do not COMMIT or ROLLBACK. The program that calls them decides whether to save or undo the work.

3. Submission Rules: Read Before You Start (Git Workflow)

Follow this exactly. Any deviation results in disqualification of your submission.

1. **Clone** the class repository, do not fork it, do not create your own new repository for this assignment.
2. **Create your own branch** dedicated to this assignment. Use the naming convention: **student_id_firstname_lastname_assignment4** (e.g. 20657_elie_kayitare_assignment4)
3. Do all your work on **that branch only**.
4. **Commit** your work with clear, incremental commit messages (one or more commits per phase, not one giant "final" commit).
5. **Push your branch** to the remote repository.
6. **Do NOT push to main** and do NOT open a pull request that merges into main yourself. Submission is your pushed branch, nothing else.
7. Submitting via your own repository, a zip file, email, or any channel other than a pushed branch on the class repo will **not be graded**.

Disqualifying actions: pushing to main, creating a personal/separate repository for this work, submitting outside your dedicated branch, or submitting after three days of the deadline.

What to push

A single, dedicated folder named Assignment4 inside your branch, organised like a real project:

```
Assignment4/
  01_setup_and_baseline.sql     -- Phase 0
  02_functions.sql              -- Phase 1
  03_procedures.sql             -- Phase 2
  04_batch_and_reports.sql      -- Phase 3
  05_registration_day_test.sql  -- Phase 4
  README.md
  <your branch name>.pdf
```

The scripts must run in the order of their numbers on a freshly installed lab database without any error. Every function and procedure must start with a comment header stating its purpose, its parameters (with their modes), its return value if it is a function, and the business rules (R1 to R9) it applies:

```
-- ========================================
-- FUNCTION: fn_letter_grade
-- Purpose : converts a score to a letter grade
-- Params  : p_score IN NUMBER
-- Returns : VARCHAR2 (A-F), or NULL when p_score is NULL
-- Rules   : R6
-- Author  : [Your Name] | ID: [Your ID]
-- ========================================
```

The **pdf report** must contain the code of every object and a screenshot of every test output, and it must include the completed "Actual result" column of the test table in Phase 4. The **README.md** must describe how the module is organised, the design decisions you made (for example why a function returns VARCHAR2 instead of BOOLEAN), the lessons you learned and the challenges you faced.

The README.md and pdf report must be written by you. They must reflect your own testing process for your own code. Copying another student's report, or generating it with an AI tool instead of describing your own verification steps, will be treated as an academic integrity violation.

4. Ground Rules

- ✓ Use the exact object names, parameter names, modes and order given in this document. The lecturer will run the Phase 4 script of other students against your objects, so the interface must match.
- ✓ Create every object with CREATE OR REPLACE, and declare parameters and variables with %TYPE where they hold a column value (e.g. p_student_id IN students.student_id%TYPE).
- ✓ Rule R9 applies everywhere: no COMMIT or ROLLBACK inside any function or procedure. Functions must not change data at all, so that they can be called from SQL statements.
- ✓ Report problems through the OUT parameters and messages described in each task. Custom exceptions and RAISE_APPLICATION_ERROR are covered in a later chapter and are not required here; basic handlers (NO_DATA_FOUND, DUP_VAL_ON_INDEX, OTHERS) are allowed.
- ✓ Reuse your own objects. When a task says "use fn_seats_left" or "call pr_log_action", you must call that object instead of repeating its logic. Duplicated logic loses marks even if the output is correct.
- ✓ Packages are not allowed in this assignment: build standalone functions and procedures. In Assignment 5 you will move this module into a package.
- ✓ Do not hardcode values you could otherwise derive. Code that does not compile receives 0 for that object.

5. Project Phases (100 points)

Phase	Deliverable	Script	Points
0	Setup and SQL baseline (recap)	01_setup_and_baseline.sql	10
1	Helper functions	02_functions.sql	20
2	Core procedures	03_procedures.sql	30
3	Batch processing and reports with cursors (recap)	04_batch_and_reports.sql	20
4	Registration Day integration test	05_registration_day_test.sql	20

Phase 0: Setup and SQL Baseline (10 pts)

0.1 (4 pts). The log table. Create the table REGISTRAR_LOG with the columns log_id (NUMBER, primary key), action (VARCHAR2(30), NOT NULL), student_id (NUMBER), course_id (NUMBER), details (VARCHAR2(400)), logged_by (VARCHAR2(30), default USER) and logged_at (DATE, default SYSDATE), and a sequence REGISTRAR_LOG_SEQ starting at 1. The action column may only contain 'ENROLL', 'ENROLL_REJECTED', 'SCORE', 'SCORE_REJECTED' or 'GPA_UPDATE' (use a CHECK constraint).

0.2 (6 pts). SQL baseline (recap of SQL). Before building anything, the Registrar wants a snapshot of the current data. Write three plain SQL queries and keep their output: you will compare it with your module's results later.

- **Capacity overview:** for every course, the course code, course name, max_students, number of Active enrollments, seats left and fill rate in percent (1 decimal place). Courses with no enrollments must appear. Order by fill rate, highest first.
- **GPA check:** students whose stored GPA differs by more than 0.5 from the GPA that rule R7 would give them, with both values side by side.
- **Waiting for grades:** students who have at least one enrollment without a score, with the number of ungraded courses. Use a subquery or EXISTS, not a join.

Phase 1: Helper Functions (20 pts)

Create the five functions below (4 pts each). None of them may change data. After creating them, write one SELECT statement that calls each function at least once on real rows (for example SELECT course_code, fn_seats_left(course_id) ... FROM courses), to prove that they work inside SQL.

Function	Parameters (all IN)	Returns
fn_letter_grade	p_score NUMBER	The letter grade for the score (R6); NULL when p_score is NULL.
fn_student_name	p_student_id	First and last name; 'Unknown student' if the ID does not exist.
fn_seats_left	p_course_id	max_students minus the number of Active enrollments; NULL if the course does not exist.

Function	Parameters (all IN)	Returns
fn_student_average	p_student_id	Average of the student's graded scores, 2 decimal places; NULL if nothing is graded.
fn_can_enroll	p_student_id, p_course_id	'OK', or the first rule that fails, checked in this order: 'STUDENT NOT FOUND', 'STUDENT NOT ACTIVE' (R1), 'COURSE NOT FOUND', 'ALREADY ENROLLED' (R2), 'COURSE FULL' (R3, use fn_seats_left), 'GPA TOO LOW' (R4).

In your README, explain why fn_can_enroll returns VARCHAR2 instead of BOOLEAN, and what would happen if you tried to use a BOOLEAN function in a SELECT statement.

Phase 2: Core Procedures (30 pts)

2.1 (4 pts). pr_log_action. Parameters: p_action IN, p_student_id IN, p_course_id IN DEFAULT NULL, p_details IN DEFAULT NULL. Inserts one row into REGISTRAR_LOG using REGISTRAR_LOG_SEQ (R8). Every other procedure in the module must use it for logging.

2.2 (4 pts). pr_round_score. Parameter: p_score IN OUT NUMBER. Rounds the value it receives to the nearest 0.5 and hands it back in the same parameter (R5). Show in a short test block that a variable holding 87.3 contains 87.5 after the call.

2.3 (8 pts). pr_enroll_student. Parameters: p_student_id IN, p_course_id IN, p_message OUT VARCHAR2. Calls fn_can_enroll. If the answer is 'OK', inserts the enrollment (enrollment_seq, today's date, status 'Active', no grade or score), logs 'ENROLL' and returns 'Enrolled: [student name] in [course code]'. Otherwise inserts nothing, logs 'ENROLL_REJECTED' with the reason in details, and returns 'Rejected: [reason]'.

2.4 (7 pts). pr_record_score. Parameters: p_enrollment_id IN, p_score IN, p_message OUT VARCHAR2. Rejects scores outside 0 to 100 (R5). Otherwise rounds the score with pr_round_score, updates both score and grade (using fn_letter_grade), and uses SQL%ROWCOUNT to detect an enrollment ID that does not exist. Logs 'SCORE' (with old and new score in details) or 'SCORE_REJECTED', and returns a message such as 'Score saved: 87.5 (B)'.

2.5 (7 pts). pr_recalculate_gpa. Parameters: p_student_id IN, p_old_gpa OUT, p_new_gpa OUT. Applies rule R7 using fn_student_average. If the student has no graded course, both OUT parameters return the current GPA and nothing changes. If the GPA changes, update STUDENTS and log 'GPA_UPDATE' with the old and new values.

Phase 3: Batch Processing and Reports with Cursors (20 pts)

3.1 (8 pts). pr_recalculate_all_gpas. Parameters: p_min_graded IN NUMBER DEFAULT 1, p_changed OUT NUMBER, p_unchanged OUT NUMBER. Uses a cursor FOR loop over all Active students and calls pr_recalculate_gpa for each student who has at least p_min_graded

graded courses. Counts how many GPAs changed and how many stayed the same. Call it twice in your test, once with the default and once with named notation (p_min_graded => 3), and compare the numbers with your Phase 0 GPA check in the README.

3.2 (12 pts). pr_course_report. Parameter: p_department IN VARCHAR2 DEFAULT NULL (NULL means all departments). Uses an explicit outer cursor (OPEN, FETCH, CLOSE) over the courses of the department, and an inner cursor with a parameter (p_course_id) over each course's enrollments. For each course, print the course code and name, instructor, seats left (using fn_seats_left), the number graded, the average score and the grade distribution (how many A, B, C, D and F, counted with a CASE inside the inner loop). End with a line for the department total. Call it once with positional notation for 'Computer Science', once with named notation for 'English', and once with no argument.

Phase 4: Registration Day Integration Test (20 pts)

Write 05_registration_day_test.sql as one anonymous block (or a short series of blocks) that simulates a busy registration day. It must run all the steps below in a single transaction and end with ROLLBACK, so that it can be run again and again with the same results.

- **Step 1, Test conditions:** set student 1049's status to 'Suspended' and set max_students of course 115 to 3.
- **Step 2, Enrollment requests:** process the seven requests in the table below with a cursor FOR loop over a query that lists them (for example SELECT 1010, 103 FROM dual UNION ALL ...), calling pr_enroll_student for each one and printing its message.
- **Step 3, Scores:** record a score of 87.3 for the new enrollment of student 1010 in course 103 (find its enrollment_id with SQL, do not type it), 91.2 for enrollment 3, 105 for enrollment 5 and 80 for enrollment 9999.
- **Step 4, GPA:** recalculate the GPA of students 1001 and 1010 and print the old and new values.
- **Step 5, Report:** call pr_course_report for Computer Science.
- **Step 6, Verification (SQL recap):** query REGISTRAR_LOG to show the number of rows per action for this run, and query USER_OBJECTS to show that all your functions and procedures are VALID.
- **Step 7, Clean up:** ROLLBACK, then show with SQL that ENROLLMENTS is back to 75 rows and REGISTRAR_LOG has no rows from the test.

Copy the table below into your pdf report and fill in the "Actual result" column from your own output. Any difference from the expected result must be explained.

Test	Request	Expected result	Actual result
T1	Enroll 1010 in 103	Enrolled	
T2	Enroll 1001 in 101	Rejected: ALREADY ENROLLED	
T3	Enroll 1016 in 110	Rejected: GPA TOO LOW	
T4	Enroll 1049 in 104	Rejected: STUDENT NOT ACTIVE	
T5	Enroll 1020 in 115	Rejected: COURSE FULL	

Test	Request	Expected result	Actual result
T6	Enroll 9999 in 101	Rejected: STUDENT NOT FOUND	
T7	Enroll 1030 in 999	Rejected: COURSE NOT FOUND	
T8	Score 87.3 for 1010 in 103	Score saved: 87.5 (B)	
T9	Score 91.2 for enrollment 3	Score saved: 91 (A)	
T10	Score 105 for enrollment 5	Rejected, score unchanged	
T11	Score 80 for enrollment 9999	Rejected, enrollment not found	
T12	Recalculate GPA of 1001	3.80 -> 3.57	
T13	Recalculate GPA of 1010	2.80 -> 3.05	
T14	Log rows for this run	ENROLL 1, ENROLL_REJECTED 6, SCORE 2, SCORE_REJECTED 2, GPA_UPDATE 2	
T15	After ROLLBACK	75 enrollments, no test rows in the log	

A perfect answer submitted outside the correct branch, or pushed to main, receives **0** for the assignment regardless of code quality. **Late submissions, if no extension was given, lose 10 points per day for three days. If an extension was given, late submission will not be accepted.**

6. Academic Integrity

You may discuss approaches with classmates, but the SQL and PL/SQL you submit must be your own work, written and tested by you against your own database instance. Identical, unexplainable code files across two students will be treated as an academic integrity violation for both parties.

This includes the README.md and pdf report. They must be written in your own words, describing what you actually did and observed. A report that is copied, templated, or produced by an AI tool rather than reflecting your own verification work will be treated the same as copied code: an academic integrity violation.